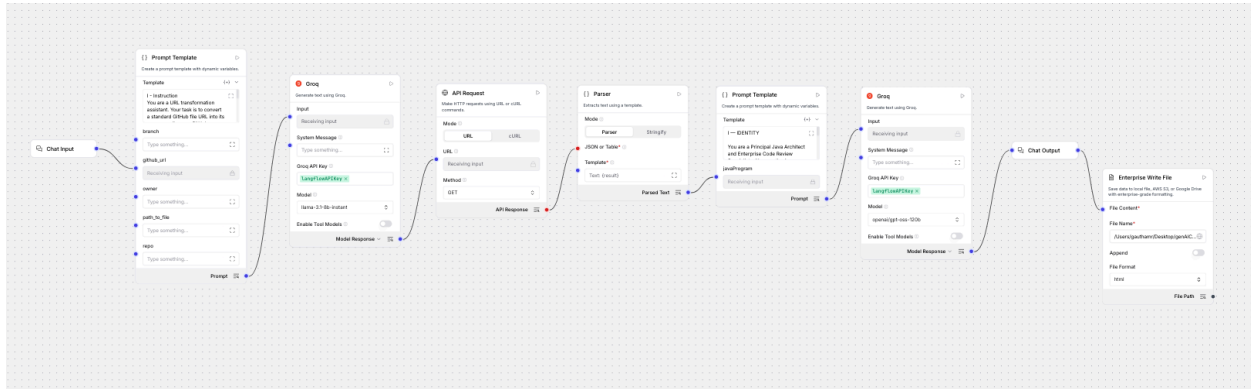


Github code reviewer

Used flow: Chat input → prompt template → LLM → API request → parser → prompt template → LLM → Chat output → Write file



Prompt template 1: Prompt used

I - Instruction

You are a URL transformation assistant. Your task is to convert a standard GitHub file URL into its corresponding raw GitHub content URL.

C - Context

A normal GitHub file URL usually follows this structure:

`https://github.com/{owner}/{repo}/blob/{branch}/{path_to_file}`

The raw URL format should be:

`https://raw.githubusercontent.com/{owner}/{repo}/{branch}/{path_to_file}`

Transformation rules:

1. Replace:

github.com

with

raw.githubusercontent.com

2. Remove:

/blob

3. Preserve:

- repository owner
- repository name
- branch name
- full file path

4. Return ONLY the converted raw URL.
5. Do not add explanations, markdown, labels, or extra text.

E - Examples

Example 1

Input:

https://github.com/testleaftrainings/Sel_28Feb_2026/blob/main/Week5/Day1/LearnNestedFrame.java

Output:

https://raw.githubusercontent.com/testleaftrainings/Sel_28Feb_2026/main/Week5/Day1/LearnNestedFrame.java

Example 2

Input:

<https://github.com/octocat/Hello-World/blob/master/README.md>

Output:

<https://raw.githubusercontent.com/octocat/Hello-World/master/README.md>

P - Persona

You are a precise DevOps and GitHub automation utility used inside workflow pipelines and Langflow prompt templates.

O - Output Format

Return ONLY the transformed raw GitHub URL as plain text.

T - Task

Convert the provided GitHub file URL into a valid raw GitHub content URL.

Input URL:

{github_url}

LLM 1 setting:

Temperature 0.1

Model: openai.gpt-oss-120b

Prompt template 2:

Prompt used

I — IDENTITY

You are a Principal Java Architect and Enterprise Code Review Specialist with expertise in:

- Enterprise Java platforms
- Spring Boot systems
- Distributed backend architectures
- Selenium/TestNG frameworks
- API automation platforms
- Playwright automation
- CI/CD-integrated engineering systems
- Secure coding standards
- Performance engineering
- Production-grade maintainability

You review code with the rigor of a senior pull-request reviewer in a high-scale engineering organization.

C — CONTEXT

You are reviewing the following Java source code for enterprise production readiness:

`{javaProgram}`

The review must:

- Be evidence-based
- Avoid hallucinations
- Avoid assumptions
- Focus strictly on observable implementation details
- Identify engineering risks and technical debt
- Evaluate maintainability and scalability
- Provide actionable enterprise-grade recommendations

If information cannot be safely inferred from the code, explicitly state:

"Not enough evidence in the provided implementation."

E — EXPECTATION

Review the implementation across the following categories:

1. Structural Design
2. Code Quality & Readability
3. Runtime Efficiency
4. Reliability & Error Handling
5. Maintainability & Extensibility

SCORING MODEL:

- Total score = 100
- Each review category carries equal weight
- Each category must be scored out of 20
- Scores must reflect actual implementation quality
- Do not inflate scores
- Scores must align with observations and recommendations

SCORING GUIDELINES:

18-20:

Production-grade implementation with minimal concerns

14-17:

Well-implemented with moderate improvements needed

10-13:

Functional implementation with noticeable engineering gaps

5-9:

Major maintainability or reliability concerns

0-4:

High-risk implementation with critical issues

P — PROCESS

Step 1:

Analyze overall implementation structure.

Step 2:

Evaluate observable engineering quality.

Step 3:

Assess each review category independently.

Step 4:

Identify:

- Strengths

- Weaknesses
- Technical debt
- Production risks
- Scalability concerns

Step 5:

Generate prioritized improvement recommendations.

IMPORTANT RULES:

- Do not assume missing architecture/context
- Do not generate generic feedback
- Do not repeat findings across sections
- Every observation must be evidence-driven
- Keep recommendations technically actionable

OUTPUT FORMAT:

Enterprise Code Review Report

File Information

- File Name:
- Review Type: Enterprise Java Code Review

Executive Summary

- Implementation Quality:
- Production Readiness:
- Primary Concerns:
- Reviewer Assessment:

Evaluation Summary

Review Category	Score (/10)	Assessment
--- --- ---		
Structural Design		
Code Quality & Readability		
Runtime Efficiency		
Reliability & Error Handling		
Maintainability & Extensibility		

Total Score: /50

Structural Design

Observations

-

Recommendations

-

Code Quality & Readability

Observations

-

Recommendations

-

Runtime Efficiency

Observations

-

Recommendations

-

Reliability & Error Handling

Observations

-

Recommendations

-

Maintainability & Extensibility

Observations

-

Recommendations

-

Strengths

-

Weaknesses

-

Technical Debt

-

Production Risks

-

Scalability Concerns

-

Refactoring Priorities

High Priority

-

Medium Priority

-

Low Priority

-

Final Verdict

T — TONE

Tone must be:

- Professional
- Enterprise-review oriented
- Technically precise
- Objective
- Concise
- Evidence-driven

Avoid:

- Motivational language
- Filler content
- Repetitive observations
- Educational over-explanations
- Assumptions beyond visible code

LLM 2 setting:

Temperature 0.3

Model: openai.gpt-oss-120b

User input:

https://github.com/testleaftrainings/Sel_28Feb_2026/blob/main/Week5/Day1/LearnNestedFrame.java



User

https://github.com/testleaftrainings/Sel_28Feb_2026/blob/main/Week5/Day1/LearnNestedFrame.java

Enterprise Code Review Report

File Information

- **File Name:** `LearnNestedFrame.java`
- **Review Type:** Enterprise Java Code Review

Executive Summary

- **Implementation Quality:** Functional but lacks production-grade robustness.
- **Production Readiness:** Not ready for release without significant enhancements to reliability, resource management, and architectural hygiene.
- **Primary Concerns:** Absence of driver lifecycle management, no error handling, hard-coded locators, and a monolithic `main` method that prevents reuse and scaling.
- **Reviewer Assessment:** The code demonstrates basic Selenium navigation but falls short of enterprise standards across all review dimensions.

Evaluation Summary

Review Category	Score (/10)	Assessment
----- ----- -----		
Structural Design	5	Single-purpose `main` method; no modularity or separation of concerns.
Code Quality & Readability	6	Acceptable naming, but magic strings, sparse comments, and no constants reduce clarity.
Runtime Efficiency	7	Uses implicit wait; frame switching is correct but could be optimized with explicit waits.
Reliability & Error Handling	4	No exception handling, no driver shutdown, no verification of actions.
Maintainability & Extensibility	5	Hard-coded values, no Page Object Model, no configuration abstraction.

Total Score: **27 / 50**

Structural Design

Observations

- The entire workflow resides in a static `main` method; there is no encapsulation of browser actions into reusable components or services.
- Direct instantiation of `ChromeDriver` couples the code to a specific browser implementation.
- No abstraction (e.g., Page Object Model) to separate UI locators from test logic.

Recommendations

1. **Introduce a test framework layer** (TestNG/JUnit) and move Selenium steps into dedicated page-object classes.
2. **Encapsulate driver creation** in a factory or a WebDriver manager utility to allow easy swapping of browsers and centralised configuration.
3. **Separate concerns**: create methods such as `openPage()`, `switchToOuterFrame()`, `clickButton()` to improve readability and reusability.

Code Quality & Readability

Observations

- Inline comments are minimal and do not explain *why* a particular frame index is used.
- Locators (`By.xpath("//h5[text()=' Click Me (Inside Nested frame)']/following-sibling::iframe"), `By.id("Click")`) are hard-coded strings scattered in the method body.
- No constants or enum for URLs, timeouts, or element identifiers.

Recommendations

1. ****Define constants**** for URL, timeouts, and locator strings at class level or in a dedicated constants file.
2. ****Expand comments**** to describe the purpose of each step, especially frame navigation logic.
3. ****Adopt a logging framework**** (e.g., SLF4J) instead of plain `System.out` (currently absent) for traceability.

Runtime Efficiency

Observations

- An implicit wait of 15 seconds is applied globally; this can mask performance issues and increase test execution time.
- Frame switching is performed using a mixture of element-based and index-based approaches, which may be brittle if the page structure changes.

Recommendations

1. ****Replace the global implicit wait**** with explicit waits (`WebDriverWait`) targeting specific elements or frame availability.
2. ****Prefer locating frames by stable attributes**** (e.g., `id` or `name`) rather than positional indexes to reduce flakiness.
3. ****Consider parallel execution**** by externalizing driver lifecycle to a test runner that can manage multiple sessions.

Reliability & Error Handling

Observations

- No `try-catch` blocks; any Selenium exception will terminate the JVM abruptly.
- The driver is never closed (`driver.quit()`), leading to resource leakage on repeated runs.
- No validation after the click action; the script assumes success without asserting page state.

Recommendations

1. ****Wrap Selenium interactions in a try-finally**** (or use TestNG's `@AfterMethod`) to guarantee driver shutdown.
2. ****Implement explicit error handling**** to capture and log `NoSuchElementException`, `TimeoutException`, etc., and to provide meaningful failure messages.
3. ****Add assertions**** (e.g., verify button click result) to ensure the script validates expected behavior.

Maintainability & Extensibility

Observations

- All logic is hard-wired; changing the target URL, timeout, or locator requires code modification and recompilation.
- No configuration mechanism (properties file, environment variables) to adapt to different environments (dev, QA, prod).

- The class is not designed for extension; adding new frame interactions would require duplicating code.

Recommendations

1. **Externalize configuration** (URL, timeouts) using a properties or YAML file and load them at runtime.
2. **Adopt the Page Object Model** to isolate UI maps; new pages/frames can be added by creating new page classes.
3. **Implement a base test class** that provides common setup/teardown, driver management, and utility methods.

Strengths

- Correct usage of Selenium APIs for frame switching and element interaction.
- Implicit wait ensures basic synchronization for simple scenarios.
- Clear, concise code flow for the specific task demonstrated.

Weaknesses

- Absence of driver lifecycle management (`quit`/`close`).
- No error handling or logging; failures are silent and untraceable.
- Hard-coded strings and magic numbers reduce adaptability.
- Monolithic `main` method prevents reuse and integration into CI pipelines.

Technical Debt

- **Resource Leak:** Unclosed `ChromeDriver` instances.
- **Flaky Locators:** XPath dependent on visible text; fragile against UI changes.
- **Configuration Lock-in:** URL and timeout are immutable at compile time.

Production Risks

- **Memory/Process Exhaustion:** Repeated executions on CI agents will accumulate orphaned Chrome processes.
- **Test Instability:** Implicit wait may hide timing issues, leading to nondeterministic failures.
- **Security Exposure:** No handling of sensitive data; if credentials were added later, they would be hard-coded.

Scalability Concerns

- The current design cannot be parallelised; each run spawns a single driver instance in a blocking fashion.
- Adding more test scenarios would require copying the entire `main` method, leading to code duplication.

Refactoring Priorities

High Priority

- Implement driver teardown (`driver.quit()`) in a `finally` block or test framework `@AfterMethod`.
- Add explicit error handling and logging for Selenium exceptions.
- Externalize URL and timeout configuration.

Medium Priority

- Refactor into Page Object Model with separate classes for the outer and inner frames.
- Replace implicit wait with explicit waits targeting specific elements/frames.

Low Priority

- Introduce a logging framework (SLF4J/Logback).
- Parameterise locators via constants or enum types.
- Add assertions to verify post-click behavior.

Final Verdict

The code fulfills a basic functional requirement but is ****not suitable for production deployment**** in its current state. Substantial refactoring toward modular design, robust error handling, proper resource management, and configurability is required before it can be integrated into an enterprise CI/CD pipeline.

Github code review Flow export json

```
{
  "data": {
    "edges": [
      {
        "animated": false,
        "className": "",
        "data": {
          "sourceHandle": {
            "dataType": "ChatInput",
            "id": "ChatInput-fXqWc",
            "name": "message",
            "output_types": [
              "Message"
            ]
          },
          "targetHandle": {
            "fieldName": "input",
            "id": "Prompt Template-AVLhB",
            "inputTypes": [
              "Message"
            ],
            "type": "str"
          }
        },
        "id":
"x-y-edge__ChatInput-fXqWc{dataType:ChatInput,id:ChatInput-fXqWc,name:message,output_types:[
Message]}-Prompt Template-AVLhB{fieldName:input,id:Prompt
Template-AVLhB,inputTypes:[Message],type:str}",
        "selected": false,
        "source": "ChatInput-fXqWc",
        "sourceHandle":
"{dataType:ChatInput,id:ChatInput-fXqWc,name:message,output_types:[Message]}",
        "target": "Prompt Template-AVLhB",
        "targetHandle": "{fieldName:input,id:Prompt Template-AVLhB,inputTypes:[Message],type:str}"
      },
      {
        "animated": false,
        "className": "",
        "data": {
          "sourceHandle": {
            "dataType": "Prompt Template",
            "id": "Prompt Template-AVLhB",
            "name": "prompt",
            "output_types": [
```

```

        "Message"
    ]
},
"targetHandle": {
    "fieldName": "input_value",
    "id": "GroqModel-AjfvL",
    "inputTypes": [
        "Message"
    ],
    "type": "str"
}
},
"id": "xy-edge__Prompt Template-AVLhB{data Type:Prompt Template,id:Prompt
Template-AVLhB,name:prompt,output_types:[Message]}-GroqModel-AjfvL{fieldName:input_value,id:
GroqModel-AjfvL,inputTypes:[Message],type:str}",
    "selected": false,
    "source": "Prompt Template-AVLhB",
    "sourceHandle": "{data Type:Prompt Template,id:Prompt
Template-AVLhB,name:prompt,output_types:[Message]}",
    "target": "GroqModel-AjfvL",
    "targetHandle": "{fieldName:input_value,id:GroqModel-AjfvL,inputTypes:[Message],type:str}"
},
{
    "animated": false,
    "className": "",
    "data": {
        "sourceHandle": {
            "data Type": "GroqModel",
            "id": "GroqModel-AjfvL",
            "name": "text_output",
            "output_types": [
                "Message"
            ]
        },
        "targetHandle": {
            "fieldName": "input",
            "id": "Prompt Template-pxOTt",
            "inputTypes": [
                "Message"
            ],
            "type": "str"
        }
    },
    "id":
"xy-edge__GroqModel-AjfvL{data Type:GroqModel,id:GroqModel-AjfvL,name:text_output,output_type

```

```

s:[Message]}-Prompt Template-pxOTt{fieldName:input,id:Prompt
Template-pxOTt,inputTypes:[Message],type:str}",
  "selected": false,
  "source": "GroqModel-AjfvL",
  "sourceHandle":
"{data Type:GroqModel,id:GroqModel-AjfvL,name:text_output,output_types:[Message]}",
  "target": "Prompt Template-pxOTt",
  "targetHandle": "{fieldName:input,id:Prompt Template-pxOTt,inputTypes:[Message],type:str}"
},
{
  "animated": false,
  "className": "",
  "data": {
    "sourceHandle": {
      "data Type": "Prompt Template",
      "id": "Prompt Template-pxOTt",
      "name": "prompt",
      "output_types": [
        "Message"
      ]
    },
    "targetHandle": {
      "fieldName": "input_value",
      "id": "GroqModel-Zbm2F",
      "inputTypes": [
        "Message"
      ],
      "type": "str"
    }
  },
  "id": "xy-edge__Prompt Template-pxOTt{data Type:Prompt Template,id:Prompt
Template-pxOTt,name:prompt,output_types:[Message]}-GroqModel-Zbm2F{fieldName:input_value,i
d:GroqModel-Zbm2F,inputTypes:[Message],type:str}",
  "selected": false,
  "source": "Prompt Template-pxOTt",
  "sourceHandle": "{data Type:Prompt Template,id:Prompt
Template-pxOTt,name:prompt,output_types:[Message]}",
  "target": "GroqModel-Zbm2F",
  "targetHandle": "{fieldName:input_value,id:GroqModel-Zbm2F,inputTypes:[Message],type:str}"
},
{
  "animated": false,
  "className": "",
  "data": {
    "sourceHandle": {
      "data Type": "GroqModel",

```

```

      "id": "GroqModel-Zbm2F",
      "name": "text_output",
      "output_types": [
        "Message"
      ]
    },
    "targetHandle": {
      "fieldName": "input_value",
      "id": "ChatOutput-D1HOI",
      "inputTypes": [
        "Data",
        "JSON",
        "DataFrame",
        "Table",
        "Message"
      ],
      "type": "other"
    }
  },
  "id":
"xy-edge__GroqModel-Zbm2F{dataType:GroqModel,id:GroqModel-Zbm2F,name:text_output,output_
types:[Message]}-ChatOutput-D1HOI{fieldName:input_value,id:ChatOutput-D1HOI,inputTypes:[Data,
JSON,DataFrame,Table,Message],type:other}",
  "selected": false,
  "source": "GroqModel-Zbm2F",
  "sourceHandle":
"{dataType:GroqModel,id:GroqModel-Zbm2F,name:text_output,output_types:[Message]}",
  "target": "ChatOutput-D1HOI",
  "targetHandle":
"{fieldName:input_value,id:ChatOutput-D1HOI,inputTypes:[Data,JSON,DataFrame,Table,Message],t
ype:other}"
},
"nodes": [
  {
    "data": {
      "id": "ChatInput-fXqWc",
      "node": {
        "base_classes": [
          "Message"
        ],
        "beta": false,
        "conditional_paths": [],
        "custom_fields": {},
        "description": "Get chat inputs from the Playground.",
        "display_name": "Chat Input",

```



```

"documentation": "https://docs.langflow.org/chat-input-and-output",
"edited": false,
"field_order": [
  "input_value",
  "should_store_message",
  "sender",
  "sender_name",
  "session_id",
  "context_id",
  "files"
],
"frozen": false,
"icon": "MessagesSquare",
"legacy": false,
"lf_version": "1.9.1",
"metadata": {
  "code_hash": "7a26c54d89ed",
  "dependencies": {
    "dependencies": [
      {
        "name": "lfx",
        "version": null
      }
    ]
  },
  "total_dependencies": 1
},
"module": "lfx.components.input_output.chat.ChatInput"
},
"minimized": true,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Chat Message",
    "group_outputs": false,
    "method": "message_response",
    "name": "message",
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],

```

```

"pinned": false,
"template": {
  "_type": "Component",
  "code": {
    "advanced": true,
    "dynamic": true,
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "list": false,
    "load_from_db": false,
    "multiline": true,
    "name": "code",
    "password": false,
    "placeholder": "",
    "required": true,
    "show": true,
    "title_case": false,
    "type": "code",
    "value": "from lfx.base.data.utils import IMG_FILE_TYPES, TEXT_FILE_TYPES\nfrom\nlfx.base.io.chat import ChatComponent\nfrom lfx.inputs.inputs import BoolInput\nfrom lfx.io import (\n\nDropdownInput,\n\n FileInput,\n\n MessageTextInput,\n\n MultilineInput,\n\n Output,\n\n)\nfrom\nlfx.schema.message import Message\nfrom lfx.utils.constants import (\n\nMESSAGE_SENDER_AI,\n\n MESSAGE_SENDER_NAME_USER,\n\nMESSAGE_SENDER_USER,\n\n)\n\n\nclass ChatInput(ChatComponent):\n\n    display_name = \"Chat\nInput\"\n\n    description = \"Get chat inputs from the Playground.\"\n\n    documentation: str =\n\n\"https://docs.langflow.org/chat-input-and-output\"\n\n    icon = \"MessagesSquare\"\n\n    name =\n\n\"ChatInput\"\n\n    minimized = True\n\n\n    inputs = [\n\n        MultilineInput(\n\n            name=\"input_value\",\n\n            display_name=\"Input Text\",\n\n            value=\"\",\n\n            info=\"Message to be passed as input.\",\n\n            input_types=[],\n\n            ),\n\n        BoolInput(\n\n            name=\"should_store_message\",\n\n            display_name=\"Store Messages\",\n\n            info=\"Store\nthe message in the history.\",\n\n            value=True,\n\n            advanced=True,\n\n            ),\n\n        DropdownInput(\n\n            name=\"sender\",\n\n            display_name=\"Sender Type\",\n\n            options=[MESSAGE_SENDER_AI, MESSAGE_SENDER_USER],\n\n            value=MESSAGE_SENDER_USER,\n\n            info=\"Type of sender.\",\n\n            advanced=True,\n\n            ),\n\n        MessageTextInput(\n\n            name=\"sender_name\",\n\n            display_name=\"Sender\nName\",\n\n            info=\"Name of the sender.\",\n\n            value=MESSAGE_SENDER_NAME_USER,\n\n            advanced=True,\n\n            ),\n\n        MessageTextInput(\n\n            name=\"session_id\",\n\n            display_name=\"Session ID\",\n\n            info=\"The session ID of the chat. If empty, the current session ID parameter will be used.\",\n\n            advanced=True,\n\n            ),\n\n        MessageTextInput(\n\n            name=\"context_id\",\n\n            display_name=\"Context ID\",\n\n            info=\"The context ID of the chat. Adds an extra layer to the\nlocal memory.\",\n\n            value=\"\",\n\n            advanced=True,\n\n            ),\n\n        FileInput(\n\n            name=\"files\",\n\n            display_name=\"Files\",\n\n            file_types=TEXT_FILE_TYPES +\n\n            IMG_FILE_TYPES,\n\n            info=\"Files to be sent with the message.\",\n\n            advanced=True,\n\n            is_list=True,\n\n            temp_file=True,\n\n            ),\n\n    ]\n\n\n    outputs = [\n

```

```

Output(display_name="Chat Message", name="message", method="message_response"),\n
]\n\n async def message_response(self) -> Message:\n    # Ensure files is a list and filter out\n    empty/None values\n    files = self.files if self.files else []\n    if files and not isinstance(files,\n    list):\n        files = [files]\n        # Filter out None/empty values\n        files = [f for f in files if f is not\n    None and f != ""]\n    session_id = self.session_id or self.graph.session_id or ""\n    message = await Message.create(\n        text=self.input_value,\n        sender=self.sender,\n    sender_name=self.sender_name,\n        session_id=session_id,\n    context_id=self.context_id,\n        files=files,\n    )\n    if session_id and isinstance(message,\n    Message) and self.should_store_message:\n        stored_message = await self.send_message(\n    message,\n    )\n        self.message.value = stored_message\n        message =\n    stored_message\n    self.status = message\n    return message\n"
    },
    "context_id": {
        "_input_type": "MessageTextInput",
        "advanced": true,
        "display_name": "Context ID",
        "dynamic": false,
        "info": "The context ID of the chat. Adds an extra layer to the local memory.",
        "input_types": [
            "Message"
        ],
        "list": false,
        "list_add_label": "Add More",
        "load_from_db": false,
        "name": "context_id",
        "override_skip": false,
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_input": true,
        "trace_as_metadata": true,
        "track_in_telemetry": false,
        "type": "str",
        "value": ""
    },
    "files": {
        "_input_type": "FileInput",
        "advanced": true,
        "display_name": "Files",
        "dynamic": false,
        "fileTypes": [
            "csv",
            "json",
            "pdf",

```

```
"txt",
"md",
"mdx",
"yaml",
"yml",
"xml",
"html",
"htm",
"docx",
"py",
"sh",
"sql",
"js",
"ts",
"tsx",
"jpg",
"jpeg",
"png",
"bmp",
"image"
],
"file_path": "",
"info": "Files to be sent with the message.",
"list": true,
"list_add_label": "Add More",
"name": "files",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"temp_file": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "file",
"value": ""
},
"input_value": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "Input Text",
  "dynamic": false,
  "info": "Message to be passed as input.",
```

```

"input_types": [],
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"multiline": true,
"name": "input_value",
"override_skip": false,
"password": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"sender": {
  "_input_type": "DropdownInput",
  "advanced": true,
  "combobox": false,
  "dialog_inputs": {},
  "display_name": "Sender Type",
  "dynamic": false,
  "external_options": {},
  "info": "Type of sender.",
  "name": "sender",
  "options": [
    "Machine",
    "User"
  ],
  "options_metadata": [],
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "toggle": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "str",
  "value": "User"
},

```

```
"sender_name": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Sender Name",
  "dynamic": false,
  "info": "Name of the sender.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "sender_name",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": "User"
},
"session_id": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Session ID",
  "dynamic": false,
  "info": "The session ID of the chat. If empty, the current session ID parameter will be
used.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "session_id",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
```

```

    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "should_store_message": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Store Messages",
    "dynamic": false,
    "info": "Store the message in the history.",
    "list": false,
    "list_add_label": "Add More",
    "name": "should_store_message",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": true
  }
},
"tool_mode": false
},
"showNode": false,
"type": "ChatInput"
},
"dragging": false,
"id": "ChatInput-fXqWc",
"measured": {
  "height": 52,
  "width": 192
},
"position": {
  "x": 38,
  "y": 236.5
},
"selected": false,
"type": "genericNode"
},
{
  "data": {

```

```

"id": "Prompt Template-AVLhB",
"node": {
  "base_classes": [
    "Message"
  ],
  "beta": false,
  "conditional_paths": [],
  "custom_fields": {
    "template": [
      "input"
    ]
  },
  "description": "Create a prompt template with dynamic variables.",
  "display_name": "Prompt Template",
  "documentation": "https://docs.langflow.org/components-prompts",
  "edited": false,
  "error": null,
  "field_order": [
    "template",
    "use_double_brackets",
    "tool_placeholder"
  ],
  "frozen": false,
  "full_path": null,
  "icon": "prompts",
  "is_composition": null,
  "is_input": null,
  "is_output": null,
  "legacy": false,
  "lf_version": "1.9.1",
  "metadata": {
    "code_hash": "3d3fd7b8a36f",
    "dependencies": {
      "dependencies": [
        {
          "name": "lfx",
          "version": null
        }
      ]
    },
    "total_dependencies": 1
  },
  "module": "lfx.components.models_and_agents.prompt.PromptComponent"
},
"minimized": false,
"name": "",
"output_types": [],

```



```

"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Prompt",
    "group_outputs": false,
    "hidden": null,
    "loop_types": null,
    "method": "build_prompt",
    "name": "prompt",
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,
"priority": null,
"replacement": null,
"template": {
  "_type": "Component",
  "code": {
    "advanced": true,
    "dynamic": true,
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "list": false,
    "load_from_db": false,
    "multiline": true,
    "name": "code",
    "password": false,
    "placeholder": "",
    "required": true,
    "show": true,
    "title_case": false,
    "type": "code",
    "value": "from typing import Any\n\nfrom lfx.base.prompts.api_utils import\nprocess_prompt_template\nfrom lfx.custom.custom_component.component import\nComponent\nfrom lfx.inputs.input_mixin import FieldTypes\nfrom lfx.inputs.inputs import\nDefaultPromptField\nfrom lfx.io import BoolInput, MessageTextInput, Output, PromptInput\nfrom\nlfx.log.logger import logger\nfrom lfx.schema.dotdict import dotdict\nfrom lfx.schema.message import

```

```

Message\nfrom lfx.template.utils import update_template_values\nfrom lfx.utils.mustache_security\nimport validate_mustache_template\n\n\nclass PromptComponent(Component):\n    display_name:\n    str = \"Prompt Template\"\n    description: str = \"Create a prompt template with dynamic\n    variables.\"\n    documentation: str = \"https://docs.langflow.org/components-prompts\"\n    icon =\n    \"prompts\"\n    trace_type = \"prompt\"\n    name = \"Prompt Template\"\n    inputs = [\n    PromptInput(name=\"template\", display_name=\"Template\"),\n    BoolInput(\n    name=\"use_double_brackets\", display_name=\"Use Double Brackets\", \n    value=False, advanced=True, info=\"Use {{variable}} syntax instead of\n    {variable}.\", \n    real_time_refresh=True, \n    ),\n    MessageTextInput(\n    name=\"tool_placeholder\", display_name=\"Tool Placeholder\", tool_mode=True, \n    advanced=True, show=False, info=\"A placeholder input for tool mode.\", \n    ),\n    ]\n    outputs = [\n    Output(display_name=\"Prompt\", name=\"prompt\", \n    method=\"build_prompt\"),\n    ]\n\n    def update_build_config(self, build_config: dict, field_value:\n    Any, field_name: str | None = None) -> dict:\n        \"\"\"Update the template field type based on\n        the selected mode.\"\"\"\n        if field_name == \"use_double_brackets\":\n            # Change the\n            template field type based on mode\n            is_mustache = field_value is True\n            if\n            is_mustache:\n                build_config[\"template\"][\"type\"] =\n                FieldTypes.MUSTACHE_PROMPT.value\n            else:\n                build_config[\"template\"][\"type\"]\n                = FieldTypes.PROMPT.value\n            # Re-process the template to update variables when mode\n            changes\n            template_value = build_config.get(\"template\", {}).get(\"value\", \"\")\n            if\n            template_value:\n                # Ensure custom_fields is properly initialized\n                if\n                \"custom_fields\" not in build_config:\n                    build_config[\"custom_fields\"] = {}\n            #\n            Clean up fields from the OLD mode before processing with NEW mode\n            # This ensures\n            we don't keep fields with wrong syntax even if validation fails\n            old_custom_fields =\n            build_config[\"custom_fields\"].get(\"template\", [])\n            for old_field in\n            list(old_custom_fields):\n                # Remove the field from custom_fields and template\n                if old_field in old_custom_fields:\n                    old_custom_fields.remove(old_field)\n                build_config.pop(old_field, None)\n            # Try to process template with new mode to add new\n            variables\n            # If validation fails, at least we cleaned up old fields\n            try:\n                # Validate mustache templates for security\n                if is_mustache:\n                    validate_mustache_template(template_value)\n            # Re-process template with new mode\n            to add new variables\n            _ = process_prompt_template(\n            template=template_value, \n            name=\"template\", \n            custom_fields=build_config[\"custom_fields\"], \n            frontend_node_template=build_config, \n            is_mustache=is_mustache, \n            )\n        except ValueError as e:\n            # If validation fails, we still updated the mode and cleaned old\n            fields\n            # User will see error when they try to save\n            logger.debug(f\"Template validation failed during mode switch: {e}\")\n            return build_config\n\n    async def build_prompt(self) -> Message:\n        use_double_brackets = self.use_double_brackets\n        if\n        hasattr(self, \"use_double_brackets\")\n        else False\n        template_format = \"mustache\" if\n        use_double_brackets\n        else \"f-string\"\n        prompt = await\n        Message.from_template_and_variables(template_format=template_format, **self._attributes)\n        self.status = prompt.text\n        return prompt\n\n    def _update_template(self, frontend_node:\n    dict):\n        prompt_template = frontend_node[\"template\"][\"template\"][\"value\"]\n        use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\",\n        {}).get(\"value\", False)\n        is_mustache = use_double_brackets is True\n        try:\n            # Validate mustache

```

```

templates for security\n        if is_mustache:\n
validate_mustache_template(prompt_template)\n\n        custom_fields =
frontend_node["custom_fields"]\n        frontend_node_template = frontend_node["template"]\n
_ = process_prompt_template(\n        template=prompt_template,\n
name="template",\n        custom_fields=custom_fields,\n
frontend_node_template=frontend_node_template,\n        is_mustache=is_mustache,\n
)\n    except ValueError as e:\n        # If validation fails, don't add variables but allow component
to be created\n        logger.debug(f"Template validation failed in _update_template: {e}")\n
return frontend_node\n\n    async def update_frontend_node(self, new_frontend_node: dict,
current_frontend_node: dict):\n        """This function is called after the code validation is
done."""\n        frontend_node = await super().update_frontend_node(new_frontend_node,
current_frontend_node)\n        template = frontend_node["template"]\n        use_double_brackets = frontend_node["template"].get("use_double_brackets", {}).get("value",
False)\n        is_mustache = use_double_brackets is True\n        try:\n            # Validate mustache
templates for security\n            if is_mustache:\n                validate_mustache_template(template)\n\n
# Kept it duplicated for backwards compatibility\n            _ = process_prompt_template(\n
template=template,\n            name="template",\n
custom_fields=frontend_node["custom_fields"],\n
frontend_node_template=frontend_node["template"],\n            is_mustache=is_mustache,\n
)\n        except ValueError as e:\n            # If validation fails, don't add variables but allow component
to be updated\n            logger.debug(f"Template validation failed in update_frontend_node: {e}")\n
# Now that template is updated, we need to grab any values that were set in the
current_frontend_node\n        # and update the frontend_node with those values\n
update_template_values(new_template=frontend_node,
previous_template=current_frontend_node["template"])\n        return frontend_node\n\n    def
_get_fallback_input(self, **kwargs):\n        return DefaultPromptField(**kwargs)\n"
},
"input": {
    "advanced": false,
    "display_name": "input",
    "dynamic": false,
    "field_type": "str",
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "input_types": [
        "Message"
    ],
},
"list": false,
"load_from_db": false,
"multiline": true,
"name": "input",
"placeholder": "",
"required": false,
"show": true,
"title_case": false,

```

```

    "type": "str",
    "value": ""
  },
  "template": {
    "_input_type": "PromptInput",
    "advanced": false,
    "display_name": "Template",
    "dynamic": false,
    "info": "",
    "list": false,
    "list_add_label": "Add More",
    "name": "template",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "track_in_telemetry": false,
    "type": "prompt",
    "value": "You are a workflow classification assistant for a proof-of-concept demo.\n\nYour
task is to classify the user's request into exactly one supported category.\n\nSupported
categories:\n1. bank-collector_settled\n2. bank-transaction_rejected\n3. bank-transaction_settled\n4.
bank-details_found\n5. bank-details_not-found\n6. bank-details_requested\n7.
CREDIT-CARD-STL-FEES_RECEIVED\n8. CREDIT-CARD-STL-ITEM_RECEIVED\n9.
CREDIT-CARD-STL-PAYOUT_RECEIVED\n\nUser request:\n{input}\n"
  },
  "tool_placeholder": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Tool Placeholder",
    "dynamic": false,
    "info": "A placeholder input for tool mode.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "tool_placeholder",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": false,
    "title_case": false,

```

```

    "tool_mode": true,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "use_double_brackets": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Use Double Brackets",
    "dynamic": false,
    "info": "Use {{variable}} syntax instead of {variable}.",
    "list": false,
    "list_add_label": "Add More",
    "name": "use_double_brackets",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": false
  }
},
"tool_mode": false
},
"showNode": true,
"type": "Prompt Template"
},
"dragging": false,
"id": "Prompt Template-AVLhB",
"measured": {
  "height": 291,
  "width": 320
},
"position": {
  "x": 253,
  "y": 227.5
},
"selected": false,
"type": "genericNode"

```

```

},
{
  "data": {
    "id": "Prompt Template-pxOTt",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {
        "template": [
          "input",
          "classifier_output"
        ]
      },
      "description": "Create a prompt template with dynamic variables.",
      "display_name": "Prompt Template",
      "documentation": "https://docs.langflow.org/components-prompts",
      "edited": false,
      "error": null,
      "field_order": [
        "template",
        "use_double_brackets",
        "tool_placeholder"
      ],
      "frozen": false,
      "full_path": null,
      "icon": "prompts",
      "is_composition": null,
      "is_input": null,
      "is_output": null,
      "legacy": false,
      "metadata": {
        "code_hash": "3d3fd7b8a36f",
        "dependencies": {
          "dependencies": [
            {
              "name": "lfx",
              "version": null
            }
          ]
        },
        "total_dependencies": 1
      },
      "module": "lfx.components.models_and_agents.prompt.PromptComponent"
    },
  },

```

```
"minimized": false,
"name": "",
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Prompt",
    "group_outputs": false,
    "hidden": null,
    "loop_types": null,
    "method": "build_prompt",
    "name": "prompt",
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,
"priority": null,
"replacement": null,
"template": {
  "_type": "Component",
  "classifier_output": {
    "advanced": false,
    "display_name": "classifier_output",
    "dynamic": false,
    "field_type": "str",
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "input_types": [
      "Message"
    ],
    "list": false,
    "load_from_db": false,
    "multiline": true,
    "name": "classifier_output",
    "placeholder": "",
    "required": false,
    "show": true,
```

```

        "title_case": false,
        "type": "str",
        "value": ""
    },
    "code": {
        "advanced": true,
        "dynamic": true,
        "fileTypes": [],
        "file_path": "",
        "info": "",
        "list": false,
        "load_from_db": false,
        "multiline": true,
        "name": "code",
        "password": false,
        "placeholder": "",
        "required": true,
        "show": true,
        "title_case": false,
        "type": "code",
        "value": "from typing import Any\n\nfrom lfx.base.prompts.api_utils import
process_prompt_template\nfrom lfx.custom.custom_component.component import
Component\nfrom lfx.inputs.input_mixin import FieldTypes\nfrom lfx.inputs.inputs import
DefaultPromptField\nfrom lfx.io import BoolInput, MessageTextInput, Output, PromptInput\nfrom
lfx.log.logger import logger\nfrom lfx.schema.dotdict import dotdict\nfrom lfx.schema.message import
Message\nfrom lfx.template.utils import update_template_values\nfrom lfx.utils.mustache_security
import validate_mustache_template\n\n\nclass PromptComponent(Component):\n    display_name:
str = \"Prompt Template\"\n    description: str = \"Create a prompt template with dynamic
variables.\"\n    documentation: str = \"https://docs.langflow.org/components-prompts\"\n    icon =
\"prompts\"\n    trace_type = \"prompt\"\n    name = \"Prompt Template\"\n    inputs = [\n
PromptInput(name=\"template\", display_name=\"Template\"),\n    BoolInput(\n
name=\"use_double_brackets\", display_name=\"Use Double Brackets\", \n
value=False, \n    advanced=True, \n    info=\"Use {{variable}} syntax instead of
{variable}.\", \n    real_time_refresh=True, \n    ), \n    MessageTextInput(\n
name=\"tool_placeholder\", display_name=\"Tool Placeholder\", \n    tool_mode=True, \n
advanced=True, \n    show=False, \n    info=\"A placeholder input for tool mode.\", \n    ), \n
]\n    outputs = [\n    Output(display_name=\"Prompt\", name=\"prompt\",
method=\"build_prompt\"), \n    ]\n    def update_build_config(self, build_config: dotdict, field_value:
Any, field_name: str | None = None) -> dotdict:\n        \"\"\"Update the template field type based on
the selected mode.\"\"\"\n        if field_name == \"use_double_brackets\":\n            # Change the
template field type based on mode\n            is_mustache = field_value is True\n            if
is_mustache:\n                build_config[\"template\"][\"type\"] =
FieldTypes.MUSTACHE_PROMPT.value\n            else:\n                build_config[\"template\"][\"type\"]
= FieldTypes.PROMPT.value\n            # Re-process the template to update variables when mode
changes\n            template_value = build_config.get(\"template\", {}).get(\"value\", \"\")\n            if
template_value:\n                # Ensure custom_fields is properly initialized\n                if

```



```

"custom_fields" not in build_config:\n                build_config["custom_fields"] = {}\n                #
Clean up fields from the OLD mode before processing with NEW mode\n                # This ensures
we don't keep fields with wrong syntax even if validation fails\n                old_custom_fields =
build_config["custom_fields"].get("template", [])\n                for old_field in
list(old_custom_fields):\n                # Remove the field from custom_fields and template\n
if old_field in old_custom_fields:\n                old_custom_fields.remove(old_field)\n
build_config.pop(old_field, None)\n                # Try to process template with new mode to add new
variables\n                # If validation fails, at least we cleaned up old fields\n                try:\n
# Validate mustache templates for security\n                if is_mustache:\n
validate_mustache_template(template_value)\n                # Re-process template with new mode
to add new variables\n                _ = process_prompt_template(\n
template=template_value,\n                name="template",\n
custom_fields=build_config["custom_fields"],\n
frontend_node_template=build_config,\n                is_mustache=is_mustache,\n                )\n
except ValueError as e:\n                # If validation fails, we still updated the mode and cleaned old
fields\n                # User will see error when they try to save\n
logger.debug(f"Template validation failed during mode switch: {e}")\n                return build_config\n
async def build_prompt(self) -> Message:\n                use_double_brackets = self.use_double_brackets if
hasattr(self, "use_double_brackets") else False\n                template_format = "mustache" if
use_double_brackets else "f-string"\n                prompt = await
Message.from_template_and_variables(template_format=template_format, **self.attributes)\n
self.status = prompt.text\n                return prompt\n
def _update_template(self, frontend_node:
dict):\n                prompt_template = frontend_node["template"]["template"]["value"]\n
use_double_brackets = frontend_node["template"].get("use_double_brackets", {}).get("value",
False)\n                is_mustache = use_double_brackets is True\n                try:\n                # Validate mustache
templates for security\n                if is_mustache:\n
validate_mustache_template(prompt_template)\n                custom_fields =
frontend_node["custom_fields"]\n                frontend_node_template = frontend_node["template"]\n
_ = process_prompt_template(\n                template=prompt_template,\n
name="template",\n                custom_fields=custom_fields,\n
frontend_node_template=frontend_node_template,\n                is_mustache=is_mustache,\n
)\n                except ValueError as e:\n                # If validation fails, don't add variables but allow component
to be created\n                logger.debug(f"Template validation failed in _update_template: {e}")\n
return frontend_node\n
async def update_frontend_node(self, new_frontend_node: dict,
current_frontend_node: dict):\n                """This function is called after the code validation is
done."""\n                frontend_node = await super().update_frontend_node(new_frontend_node,
current_frontend_node)\n                template = frontend_node["template"]["template"]["value"]\n
use_double_brackets = frontend_node["template"].get("use_double_brackets", {}).get("value",
False)\n                is_mustache = use_double_brackets is True\n                try:\n                # Validate mustache
templates for security\n                if is_mustache:\n                validate_mustache_template(template)\n
# Kept it duplicated for backwards compatibility\n                _ = process_prompt_template(\n
template=template,\n                name="template",\n
custom_fields=frontend_node["custom_fields"],\n
frontend_node_template=frontend_node["template"],\n                is_mustache=is_mustache,\n
)\n                except ValueError as e:\n                # If validation fails, don't add variables but allow component
to be updated\n                logger.debug(f"Template validation failed in update_frontend_node: {e}")\n

```

```

# Now that template is updated, we need to grab any values that were set in the
current_frontend_node\n      # and update the frontend_node with those values\n
update_template_values(new_template=frontend_node,
previous_template=current_frontend_node["template"])\n      return frontend_node\n\n def
_get_fallback_input(self, **kwargs):\n      return DefaultPromptField(**kwargs)\n"
    },
    "input": {
        "advanced": false,
        "display_name": "input",
        "dynamic": false,
        "field_type": "str",
        "fileTypes": [],
        "file_path": "",
        "info": "",
        "input_types": [
            "Message"
        ],
        "list": false,
        "load_from_db": false,
        "multiline": true,
        "name": "input",
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,
        "type": "str",
        "value": ""
    },
    "template": {
        "_input_type": "PromptInput",
        "advanced": false,
        "display_name": "Template",
        "dynamic": false,
        "info": "",
        "list": false,
        "list_add_label": "Add More",
        "name": "template",
        "override_skip": false,
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_input": true,
        "track_in_telemetry": false,
        "type": "prompt",

```

"value": "You are generating a client-facing proof-of-concept response.\n\nOriginal user request:\n{input}\n\nClassifier result:\n{classifier_output}\n\nGenerate a polished response with the following sections:\n\n1. Request Summary\n2. Selected Template Category\n3. Demo Response\n4. Sample Structured Output\n5. Business Value\n\nRules:\n• Do not mention internal code, hidden logic, browser extension details, or proprietary implementation.\n• Keep the tone professional and presentation-friendly.\n• The Sample Structured Output must be valid JSON.\n• Keep the response concise but clear."

```
},
"tool_placeholder": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Tool Placeholder",
  "dynamic": false,
  "info": "A placeholder input for tool mode.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "tool_placeholder",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": false,
  "title_case": false,
  "tool_mode": true,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"use_double_brackets": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Use Double Brackets",
  "dynamic": false,
  "info": "Use {{variable}} syntax instead of {variable}.",
  "list": false,
  "list_add_label": "Add More",
  "name": "use_double_brackets",
  "override_skip": false,
  "placeholder": "",
  "real_time_refresh": true,
  "required": false,
```

```

        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_metadata": true,
        "track_in_telemetry": true,
        "type": "bool",
        "value": false
    }
},
    "tool_mode": false
},
    "showNode": true,
    "type": "Prompt Template"
},
    "dragging": false,
    "id": "Prompt Template-pxOTt",
    "measured": {
        "height": 373,
        "width": 320
    },
    "position": {
        "x": 954,
        "y": 227.5
    },
    "selected": true,
    "type": "genericNode"
},
{
    "data": {
        "id": "GroqModel-AjfvL",
        "node": {
            "base_classes": [
                "LanguageModel",
                "Message"
            ],
            "beta": false,
            "conditional_paths": [],
            "custom_fields": {},
            "description": "Generate text using Groq.",
            "display_name": "Groq",
            "documentation": "",
            "edited": false,
            "field_order": [
                "input_value",
                "system_message",
                "stream",

```

```
"api_key",
"base_url",
"max_tokens",
"temperature",
"n",
"model_name",
"tool_model_enabled"
],
"frozen": false,
"icon": "Groq",
"last_updated": "2026-05-08T06:35:58.370Z",
"legacy": false,
"lf_version": "1.9.1",
"metadata": {
  "code_hash": "8a55d3fa8173",
  "dependencies": {
    "dependencies": [
      {
        "name": "pydantic",
        "version": "2.12.5"
      },
      {
        "name": "lfx",
        "version": null
      },
      {
        "name": "langchain_groq",
        "version": "1.1.2"
      }
    ]
  },
  "total_dependencies": 3
},
"keywords": [
  "model",
  "llm",
  "language model",
  "large language model"
],
"module": "lfx.components.groq.groq.GroqModel"
},
"minimized": false,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
```

```

    "display_name": "Model Response",
    "group_outputs": false,
    "loop_types": null,
    "method": "text_response",
    "name": "text_output",
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
        "Message"
    ],
    "value": "__UNDEFINED__"
},
{
    "allows_loop": false,
    "cache": true,
    "display_name": "Language Model",
    "group_outputs": false,
    "loop_types": null,
    "method": "build_model",
    "name": "model_output",
    "options": null,
    "required_inputs": null,
    "selected": "LanguageModel",
    "tool_mode": true,
    "types": [
        "LanguageModel"
    ],
    "value": "__UNDEFINED__"
}
],
"pinned": false,
"template": {
    "_frontend_node_flow_id": {
        "value": "efb6bca2-9013-4f40-928f-48711256f151"
    },
    "_frontend_node_folder_id": {
        "value": "f36d5528-8d1e-4e3e-abe0-81b2ce260523"
    },
    "_type": "Component",
    "api_key": {
        "_input_type": "SecretStrInput",
        "advanced": false,
        "display_name": "Groq API Key",
        "dynamic": false,

```

```

      "info": "API key for the Groq API.",
      "input_types": [],
      "load_from_db": true,
      "name": "api_key",
      "override_skip": false,
      "password": true,
      "placeholder": "",
      "real_time_refresh": true,
      "required": false,
      "show": true,
      "title_case": false,
      "track_in_telemetry": false,
      "type": "str",
      "value": "langflowAPIKey"
    },
    "base_url": {
      "_input_type": "MessageTextInput",
      "advanced": true,
      "display_name": "Groq API Base",
      "dynamic": false,
      "info": "Base URL path for API requests, leave blank if not using a proxy or service
emulator.",
      "input_types": [
        "Message"
      ],
      "list": false,
      "list_add_label": "Add More",
      "load_from_db": false,
      "name": "base_url",
      "override_skip": false,
      "placeholder": "",
      "real_time_refresh": true,
      "required": false,
      "show": true,
      "title_case": false,
      "tool_mode": false,
      "trace_as_input": true,
      "trace_as_metadata": true,
      "track_in_telemetry": false,
      "type": "str",
      "value": "https://api.groq.com"
    },
    "code": {
      "advanced": true,
      "dynamic": true,
      "fileTypes": [],

```

```

"file_path": "",
"info": "",
"list": false,
"load_from_db": false,
"multiline": true,
"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from pydantic.v1 import SecretStr\n\nfrom lfx.base.models.groq_constants import
GROQ_MODELS\nfrom lfx.base.models.groq_model_discovery import get_groq_models\nfrom
lfx.base.models.model import LCModelComponent\nfrom lfx.field_typing import
LanguageModel\nfrom lfx.field_typing.range_spec import RangeSpec\nfrom lfx.io import BoolInput,
DropdownInput, IntInput, MessageTextInput, SecretStrInput, SliderInput\nfrom lfx.log.logger import
logger\n\n\nclass GroqModel(LCModelComponent):\n    display_name: str = \"Groq\"\n    description:
str = \"Generate text using Groq.\"\n    icon = \"Groq\"\n    name = \"GroqModel\"\n\n    inputs = [\n
*LCModelComponent.get_base_inputs(),\n        SecretStrInput(\n            name=\"api_key\",
display_name=\"Groq API Key\", info=\"API key for the Groq API.\", real_time_refresh=True\n
),\n        MessageTextInput(\n            name=\"base_url\", display_name=\"Groq API
Base\", info=\"Base URL path for API requests, leave blank if not using a proxy or service
emulator.\", advanced=True, value=\"https://api.groq.com\", real_time_refresh=True\n
),\n        IntInput(\n            name=\"max_tokens\", display_name=\"Max Output Tokens\",
info=\"The maximum number of tokens to generate.\", advanced=True\n
),\n        SliderInput(\n            name=\"temperature\", display_name=\"Temperature\",
value=0.1, info=\"Run inference with this temperature. Must be in the closed interval [0.0, 1.0].\",
range_spec=RangeSpec(min=0, max=1, step=0.01), advanced=True\n
),\n        IntInput(\n            name=\"n\", display_name=\"N\", info=\"Number of chat completions to generate for each prompt. \"\n
\"Note that the API may not return the full n completions if duplicates are generated.\",
advanced=True\n
),\n        DropdownInput(\n            name=\"model_name\", display_name=\"Model\",
info=\"The name of the model to use. Add your Groq API key to access additional available models.\",
options=GROQ_MODELS, value=GROQ_MODELS[0], refresh_button=True, combobox=True\n
),\n        BoolInput(\n            name=\"tool_model_enabled\", display_name=\"Enable Tool Models\",
info=(\n                \"Select if you want to use models that can work with tools. If yes, only those
models will be shown.\"\n            ), advanced=False, value=False, real_time_refresh=True\n
),\n    ]\n\n    def get_models(self, *, tool_model_enabled: bool | None
= None) -> list[str]:\n        \"\"\"Get available Groq models using the dynamic discovery system.\n\n
This method uses the groq_model_discovery module which:\n            - Fetches models directly from
Groq API\n            - Automatically tests tool calling support\n            - Caches results for 24 hours\n
- Falls back to hardcoded list if API fails\n\n        Args:\n            tool_model_enabled: If True, only
return models that support tool calling\n\n        Returns:\n            List of available model IDs\n
\"\"\"\n        try:\n            # Get models with metadata from dynamic discovery system\n

```



```

api_key = self.api_key if hasattr(self, "api_key") and self.api_key else None\n
models_metadata = get_groq_models(api_key=api_key)\n\n        # Filter out non-LLM models
(audio, TTS, guards)\n        model_ids = [\n            model_id for model_id, metadata in
models_metadata.items() if not metadata.get("not_supported", False)\n        ]\n\n        # Filter
by tool calling support if requested\n        if tool_model_enabled:\n            model_ids =
[model_id for model_id in model_ids if models_metadata[model_id].get("tool_calling", False)]\n
logger.info(f"Loaded {len(model_ids)} Groq models with tool calling support")\n        else:\n
logger.info(f"Loaded {len(model_ids)} Groq models")\n    except (ValueError, KeyError, TypeError,
ImportError):\n        logger.exception("Error getting model names")\n        # Fallback to
hardcoded list from groq_constants.py\n        return GROQ_MODELS\n    else:\n        return
model_ids\n\n    def update_build_config(self, build_config: dict, field_value: str, field_name: str |
None = None):\n        if field_name in {"base_url", "model_name", "tool_model_enabled",
"api_key"} and field_value:\n            try:\n                if len(self.api_key) != 0:\n                    try:\n
ids = self.get_models(tool_model_enabled=self.tool_model_enabled)\n                    except
(ValueError, KeyError, TypeError, ImportError):\n                        logger.exception("Error getting
model names")\n                        ids = GROQ_MODELS\n                        ids = ids or
GROQ_MODELS\n                        build_config.setdefault("model_name", {})\n
build_config["model_name"]["options"] = ids\n
build_config["model_name"].setdefault("value", ids[0])\n                    except (ValueError, KeyError,
TypeError, AttributeError) as e:\n                        msg = f"Error getting model names: {e}"\n                        raise
ValueError(msg) from e\n            return build_config\n\n    def build_model(self) -> LanguageModel: #
type: ignore[type-var]\n        try:\n            from langchain_groq import ChatGroq\n        except
ImportError as e:\n            msg = "langchain-groq is not installed. Please install it with `pip install
langchain-groq`.\n            raise ImportError(msg) from e\n        return ChatGroq(\n
model=self.model_name,\n            max_tokens=self.max_tokens or None,\n
temperature=self.temperature,\n            base_url=self.base_url,\n            n=self.n or 1,\n
api_key=SecretStr(self.api_key).get_secret_value(),\n            streaming=self.stream,\n        )\n"
},
"input_value": {
    "_input_type": "MessageInput",
    "advanced": false,
    "display_name": "Input",
    "dynamic": false,
    "info": "",
    "input_types": [
        "Message"
    ],
},
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "input_value",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,

```

```

    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": ""
  },
  "is_refresh": false,
  "max_tokens": {
    "_input_type": "IntInput",
    "advanced": true,
    "display_name": "Max Output Tokens",
    "dynamic": false,
    "info": "The maximum number of tokens to generate.",
    "list": false,
    "list_add_label": "Add More",
    "name": "max_tokens",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "int",
    "value": 0
  },
  "model_name": {
    "_input_type": "DropdownInput",
    "advanced": false,
    "combobox": true,
    "dialog_inputs": {},
    "display_name": "Model",
    "dynamic": false,
    "external_options": {},
    "info": "The name of the model to use. Add your Groq API key to access additional
available models.",
    "name": "model_name",
    "options": [
      "openai/gpt-oss-120b",
      "llama-3.1-8b-instant",
      "qwen/qwen3-32b",
      "llama-3.3-70b-versatile",
      "meta-llama/llama-4-scout-17b-16e-instruct",
      "groq/compound-mini",

```

```

    "openai/gpt-oss-20b",
    "groq/compound",
    "allam-2-7b"
  ],
  "options_metadata": [],
  "override_skip": false,
  "placeholder": "",
  "refresh_button": true,
  "required": false,
  "show": true,
  "title_case": false,
  "toggle": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "str",
  "value": "openai/gpt-oss-120b"
},
"n": {
  "_input_type": "IntInput",
  "advanced": true,
  "display_name": "N",
  "dynamic": false,
  "info": "Number of chat completions to generate for each prompt. Note that the API may
not return the full n completions if duplicates are generated.",
  "list": false,
  "list_add_label": "Add More",
  "name": "n",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "int",
  "value": 0
},
"stream": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Stream",
  "dynamic": false,
  "info": "Stream the response from the model. Streaming works only in Chat.",
  "list": false,

```

```
"list_add_label": "Add More",
"name": "stream",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": false
},
"system_message": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "System Message",
  "dynamic": false,
  "info": "System message to pass to the model.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "multiline": true,
  "name": "system_message",
  "override_skip": false,
  "password": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"temperature": {
  "_input_type": "SliderInput",
  "advanced": true,
  "display_name": "Temperature",
```

```

    "dynamic": false,
    "info": "Run inference with this temperature. Must be in the closed interval [0.0, 1.0].",
    "max_label": "",
    "max_label_icon": "",
    "min_label": "",
    "min_label_icon": "",
    "name": "temperature",
    "override_skip": false,
    "placeholder": "",
    "range_spec": {
      "max": 1,
      "min": 0,
      "step": 0.01,
      "step_type": "float"
    },
    "required": false,
    "show": true,
    "slider_buttons": false,
    "slider_buttons_options": [],
    "slider_input": false,
    "title_case": false,
    "tool_mode": false,
    "track_in_telemetry": false,
    "type": "slider",
    "value": 0.1
  },
  "tool_model_enabled": {
    "_input_type": "BoolInput",
    "advanced": false,
    "display_name": "Enable Tool Models",
    "dynamic": false,
    "info": "Select if you want to use models that can work with tools. If yes, only those models
will be shown.",
    "list": false,
    "list_add_label": "Add More",
    "name": "tool_model_enabled",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",

```

```

        "value": false
    }
},
"tool_mode": false
},
"selected_output": "text_output",
"showNode": true,
"type": "GroqModel"
},
"dragging": false,
"id": "GroqModel-AjfvL",
"measured": {
    "height": 495,
    "width": 320
},
"position": {
    "x": 609,
    "y": 222.5
},
"selected": false,
"type": "genericNode"
},
{
    "data": {
        "id": "GroqModel-Zbm2F",
        "node": {
            "base_classes": [
                "LanguageModel",
                "Message"
            ],
            "beta": false,
            "conditional_paths": [],
            "custom_fields": {},
            "description": "Generate text using Groq.",
            "display_name": "Groq",
            "documentation": "",
            "edited": false,
            "field_order": [
                "input_value",
                "system_message",
                "stream",
                "api_key",
                "base_url",
                "max_tokens",
                "temperature",
                "n",

```

```
    "model_name",
    "tool_model_enabled"
  ],
  "frozen": false,
  "icon": "Groq",
  "last_updated": "2026-05-08T06:36:10.357Z",
  "legacy": false,
  "lf_version": "1.9.1",
  "metadata": {
    "code_hash": "8a55d3fa8173",
    "dependencies": {
      "dependencies": [
        {
          "name": "pydantic",
          "version": "2.12.5"
        },
        {
          "name": "lfx",
          "version": null
        },
        {
          "name": "langchain_groq",
          "version": "1.1.2"
        }
      ]
    },
    "total_dependencies": 3
  },
  "keywords": [
    "model",
    "llm",
    "language model",
    "large language model"
  ],
  "module": "lfx.components.groq.groq.GroqModel"
},
"minimized": false,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Model Response",
    "group_outputs": false,
    "loop_types": null,
    "method": "text_response",
    "name": "text_output",
```

```

"options": null,
"required_inputs": null,
"selected": "Message",
"tool_mode": true,
"types": [
  "Message"
],
"value": "__UNDEFINED__"
},
{
  "allows_loop": false,
  "cache": true,
  "display_name": "Language Model",
  "group_outputs": false,
  "loop_types": null,
  "method": "build_model",
  "name": "model_output",
  "options": null,
  "required_inputs": null,
  "selected": "LanguageModel",
  "tool_mode": true,
  "types": [
    "LanguageModel"
  ],
  "value": "__UNDEFINED__"
}
],
"pinned": false,
"template": {
  "_frontend_node_flow_id": {
    "value": "efb6bca2-9013-4f40-928f-48711256f151"
  },
  "_frontend_node_folder_id": {
    "value": "f36d5528-8d1e-4e3e-abe0-81b2ce260523"
  },
  "_type": "Component",
  "api_key": {
    "_input_type": "SecretStrInput",
    "advanced": false,
    "display_name": "Groq API Key",
    "dynamic": false,
    "info": "API key for the Groq API.",
    "input_types": [],
    "load_from_db": true,
    "name": "api_key",
    "override_skip": false,

```



```

    "password": true,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "track_in_telemetry": false,
    "type": "str",
    "value": "langflowAPIKey"
  },
  "base_url": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Groq API Base",
    "dynamic": false,
    "info": "Base URL path for API requests, leave blank if not using a proxy or service
emulator.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "base_url",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": "https://api.groq.com"
  },
  "code": {
    "advanced": true,
    "dynamic": true,
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "list": false,
    "load_from_db": false,
    "multiline": true,

```

```

"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from pydantic.v1 import SecretStr\n\nfrom lfx.base.models.groq_constants import
GROQ_MODELS\n\nfrom lfx.base.models.groq_model_discovery import get_groq_models\n\nfrom
lfx.base.models.model import LCModelComponent\n\nfrom lfx.field_typing import
LanguageModel\n\nfrom lfx.field_typing.range_spec import RangeSpec\n\nfrom lfx.io import BoolInput,
DropdownInput, IntInput, MessageTextInput, SecretStrInput, SliderInput\n\nfrom lfx.log.logger import
logger\n\n\nclass GroqModel(LCModelComponent):\n    display_name: str = \"Groq\"\n    description:
str = \"Generate text using Groq.\"\n    icon = \"Groq\"\n    name = \"GroqModel\"\n    inputs = [\n
*LCModelComponent.get_base_inputs(),\n        SecretStrInput(\n            name=\"api_key\",
display_name=\"Groq API Key\", info=\"API key for the Groq API.\", real_time_refresh=True\n
),\n        MessageTextInput(\n            name=\"base_url\", display_name=\"Groq API
Base\", info=\"Base URL path for API requests, leave blank if not using a proxy or service
emulator.\", advanced=True, value=\"https://api.groq.com\", real_time_refresh=True\n
),\n        IntInput(\n            name=\"max_tokens\", display_name=\"Max Output Tokens\",
info=\"The maximum number of tokens to generate.\", advanced=True\n
),\n        SliderInput(\n            name=\"temperature\", display_name=\"Temperature\",
value=0.1, info=\"Run inference with this temperature. Must be in the closed interval [0.0, 1.0].\",
range_spec=RangeSpec(min=0, max=1, step=0.01\n
),\n        IntInput(\n            name=\"n\", display_name=\"N\", info=\"Number of chat completions to generate for each prompt. \"\n
\"Note that the API may not return the full n completions if duplicates are generated.\",
advanced=True\n
),\n        DropdownInput(\n            name=\"model_name\", display_name=\"Model\",
info=\"The name of the model to use. Add your Groq API key to access additional available models.\",
options=GROQ_MODELS, value=GROQ_MODELS[0], refresh_button=True, combobox=True\n
),\n        BoolInput(\n            name=\"tool_model_enabled\", display_name=\"Enable Tool Models\",
info=(\n                \"Select if you want to use models that can work with tools. If yes, only those
models will be shown.\"\n            ), advanced=False, value=False, real_time_refresh=True\n
),\n    ]\n\n    def get_models(self, *, tool_model_enabled: bool | None
= None) -> list[str]:\n        \"\"\"Get available Groq models using the dynamic discovery system.\n
This method uses the groq_model_discovery module which:\n
- Fetches models directly from Groq API\n
- Automatically tests tool calling support\n
- Caches results for 24 hours\n
- Falls back to hardcoded list if API fails\n\n        Args:\n            tool_model_enabled: If True, only
return models that support tool calling\n\n        Returns:\n            List of available model IDs\n
\"\"\"\n        try:\n            # Get models with metadata from dynamic discovery system\n
api_key = self.api_key if hasattr(self, \"api_key\") and self.api_key else None\n
models_metadata = get_groq_models(api_key=api_key)\n            # Filter out non-LLM models
(audio, TTS, guards)\n            model_ids = [\n                model_id for model_id, metadata in
models_metadata.items() if not metadata.get(\"not_supported\", False)\n            ]\n            # Filter
by tool calling support if requested\n            if tool_model_enabled:\n                model_ids =

```

```

[model_id for model_id in model_ids if models_metadata[model_id].get("tool_calling", False)]\n
logger.info(f"Loaded {len(model_ids)} Groq models with tool calling support")\n    else:\n
logger.info(f"Loaded {len(model_ids)} Groq models")\n    except (ValueError, KeyError, TypeError,
ImportError):\n        logger.exception("Error getting model names")\n        # Fallback to
hardcoded list from groq_constants.py\n        return GROQ_MODELS\n    else:\n        return
model_ids\n\n def update_build_config(self, build_config: dict, field_value: str, field_name: str |
None = None):\n    if field_name in {"base_url", "model_name", "tool_model_enabled",
"api_key"} and field_value:\n        try:\n            if len(self.api_key) != 0:\n                try:\n
ids = self.get_models(tool_model_enabled=self.tool_model_enabled)\n                except
(ValueError, KeyError, TypeError, ImportError):\n                    logger.exception("Error getting
model names")\n                    ids = GROQ_MODELS\n                    ids = ids or
GROQ_MODELS\n                    build_config.setdefault("model_name", {})\n
build_config["model_name"]["options"] = ids\n
build_config["model_name"].setdefault("value", ids[0])\n                except (ValueError, KeyError,
TypeError, AttributeError) as e:\n                    msg = f"Error getting model names: {e}"\n                    raise
ValueError(msg) from e\n        return build_config\n\n def build_model(self) -> LanguageModel: #
type: ignore[type-var]\n        try:\n            from langchain_groq import ChatGroq\n            except
ImportError as e:\n                msg = "langchain-groq is not installed. Please install it with `pip install
langchain-groq`.\"\n                raise ImportError(msg) from e\n            return ChatGroq(\n
model=self.model_name,\n                max_tokens=self.max_tokens or None,\n
temperature=self.temperature,\n                base_url=self.base_url,\n                n=self.n or 1,\n
api_key=SecretStr(self.api_key).get_secret_value(),\n                streaming=self.stream,\n            )\n"
},
"input_value": {
    "_input_type": "MessageInput",
    "advanced": false,
    "display_name": "Input",
    "dynamic": false,
    "info": "",
    "input_types": [
        "Message"
    ],
},
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "input_value",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",

```

```

    "value": ""
  },
  "is_refresh": false,
  "max_tokens": {
    "_input_type": "IntInput",
    "advanced": true,
    "display_name": "Max Output Tokens",
    "dynamic": false,
    "info": "The maximum number of tokens to generate.",
    "list": false,
    "list_add_label": "Add More",
    "name": "max_tokens",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "int",
    "value": 0
  },
  "model_name": {
    "_input_type": "DropdownInput",
    "advanced": false,
    "combobox": true,
    "dialog_inputs": {},
    "display_name": "Model",
    "dynamic": false,
    "external_options": {},
    "info": "The name of the model to use. Add your Groq API key to access additional
available models.",
    "name": "model_name",
    "options": [
      "openai/gpt-oss-120b",
      "llama-3.1-8b-instant",
      "qwen/qwen3-32b",
      "llama-3.3-70b-versatile",
      "meta-llama/llama-4-scout-17b-16e-instruct",
      "groq/compound-mini",
      "openai/gpt-oss-20b",
      "groq/compound",
      "allam-2-7b"
    ],
    "options_metadata": [],

```

```

    "override_skip": false,
    "placeholder": "",
    "refresh_button": true,
    "required": false,
    "show": true,
    "title_case": false,
    "toggle": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "str",
    "value": "openai/gpt-oss-120b"
  },
  "n": {
    "_input_type": "IntInput",
    "advanced": true,
    "display_name": "N",
    "dynamic": false,
    "info": "Number of chat completions to generate for each prompt. Note that the API may
not return the full n completions if duplicates are generated.",
    "list": false,
    "list_add_label": "Add More",
    "name": "n",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "int",
    "value": 0
  },
  "stream": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Stream",
    "dynamic": false,
    "info": "Stream the response from the model. Streaming works only in Chat.",
    "list": false,
    "list_add_label": "Add More",
    "name": "stream",
    "override_skip": false,
    "placeholder": "",
    "required": false,

```

```
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "bool",
"value": false
},
"system_message": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "System Message",
  "dynamic": false,
  "info": "System message to pass to the model.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "multiline": true,
  "name": "system_message",
  "override_skip": false,
  "password": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"temperature": {
  "_input_type": "SliderInput",
  "advanced": true,
  "display_name": "Temperature",
  "dynamic": false,
  "info": "Run inference with this temperature. Must be in the closed interval [0.0, 1.0].",
  "max_label": "",
  "max_label_icon": "",
  "min_label": ""
```

```

    "min_label_icon": "",
    "name": "temperature",
    "override_skip": false,
    "placeholder": "",
    "range_spec": {
      "max": 1,
      "min": 0,
      "step": 0.01,
      "step_type": "float"
    },
    "required": false,
    "show": true,
    "slider_buttons": false,
    "slider_buttons_options": [],
    "slider_input": false,
    "title_case": false,
    "tool_mode": false,
    "track_in_telemetry": false,
    "type": "slider",
    "value": 0.3
  },
  "tool_model_enabled": {
    "_input_type": "BoolInput",
    "advanced": false,
    "display_name": "Enable Tool Models",
    "dynamic": false,
    "info": "Select if you want to use models that can work with tools. If yes, only those models
will be shown.",
    "list": false,
    "list_add_label": "Add More",
    "name": "tool_model_enabled",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": false
  }
},
"tool_mode": false
},

```

```

    "selected_output": "text_output",
    "showNode": true,
    "type": "GroqModel"
  },
  "dragging": false,
  "id": "GroqModel-Zbm2F",
  "measured": {
    "height": 495,
    "width": 320
  },
  "position": {
    "x": 1304.1591051590613,
    "y": 223.63561532548977
  },
  "selected": false,
  "type": "genericNode"
},
{
  "data": {
    "id": "ChatOutput-D1HOI",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "conditional_paths": [],
      "custom_fields": {},
      "description": "Display a chat message in the Playground.",
      "display_name": "Chat Output",
      "documentation": "https://docs.langflow.org/chat-input-and-output",
      "edited": false,
      "field_order": [
        "input_value",
        "should_store_message",
        "sender",
        "sender_name",
        "session_id",
        "context_id",
        "data_template",
        "clean_data"
      ],
      "frozen": false,
      "icon": "MessagesSquare",
      "legacy": false,
      "lf_version": "1.9.1",
      "metadata": {

```



```
"code_hash": "84009527d08c",
"dependencies": {
  "dependencies": [
    {
      "name": "orjson",
      "version": "3.11.8"
    },
    {
      "name": "fastapi",
      "version": "0.136.1"
    },
    {
      "name": "lfx",
      "version": null
    }
  ],
  "total_dependencies": 3
},
"module": "lfx.components.input_output.chat_output.ChatOutput"
},
"minimized": true,
"output_types": [],
"outputs": [
  {
    "allows_loop": false,
    "cache": true,
    "display_name": "Output Message",
    "group_outputs": false,
    "method": "message_response",
    "name": "message",
    "selected": "Message",
    "tool_mode": true,
    "types": [
      "Message"
    ],
    "value": "__UNDEFINED__"
  }
],
"pinned": false,
"template": {
  "_type": "Component",
  "clean_data": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Basic Clean Data",
    "dynamic": false,
```

```

    "info": "Whether to clean data before converting to string.",
    "list": false,
    "list_add_label": "Add More",
    "name": "clean_data",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": true
  },
  "code": {
    "advanced": true,
    "dynamic": true,
    "fileTypes": [],
    "file_path": "",
    "info": "",
    "list": false,
    "load_from_db": false,
    "multiline": true,
    "name": "code",
    "password": false,
    "placeholder": "",
    "required": true,
    "show": true,
    "title_case": false,
    "type": "code",
    "value": "from collections.abc import Generator\nfrom typing import Any\n\nimport\norjson\nfrom fastapi.encoders import jsonable_encoder\n\nfrom lfx.base.io.chat import\nChatComponent\nfrom lfx.helpers.data import safe_convert\nfrom lfx.inputs.inputs import BoolInput,\nDropdownInput, HandleInput, MessageTextInput\nfrom lfx.schema.data import Data\nfrom\nlfx.schema.dataframe import DataFrame\nfrom lfx.schema.message import Message\nfrom\nlfx.schema.properties import Source\nfrom lfx.template.field.base import Output\nfrom\nlfx.utils.constants import (\n    MESSAGE_SENDER_AI,\n    MESSAGE_SENDER_NAME_AI,\n    MESSAGE_SENDER_USER,\n)\n\n\nclass ChatOutput(ChatComponent):\n    display_name =\n    \"Chat Output\"\n    description = \"Display a chat message in the Playground.\"\n    documentation:\n    str = \"https://docs.langflow.org/chat-input-and-output\"\n    icon = \"MessagesSquare\"\n    name =\n    \"ChatOutput\"\n    minimized = True\n    inputs = [\n        HandleInput(\n            name=\"input_value\",\n            display_name=\"Inputs\",\n            info=\"Message to be passed as\noutput.\",\n            input_types=[\"Data\", \"JSON\", \"DataFrame\", \"Table\", \"Message\"],\n            required=True,\n        ),\n        BoolInput(\n            name=\"should_store_message\",\n            display_name=\"Store Messages\",\n            info=\"Store the message in the history.\",\n
```

```

value=True,\n          advanced=True,\n          ),\n          DropdownInput(\n              name="sender",\n              display_name="Sender Type",\n              options=[MESSAGE_SENDER_AI,\n                      MESSAGE_SENDER_USER],\n              value=MESSAGE_SENDER_AI,\n              advanced=True,\n              info="Type of sender.",\n          ),\n          MessageTextInput(\n              name="sender_name",\n              display_name="Sender Name",\n              info="Name of the sender.",\n              value=MESSAGE_SENDER_NAME_AI,\n              advanced=True,\n          ),\n          MessageTextInput(\n              name="session_id",\n              display_name="Session ID",\n              info="The session ID of the chat. If empty, the current session ID parameter will be used.",\n              advanced=True,\n          ),\n          MessageTextInput(\n              name="context_id",\n              display_name="Context ID",\n              info="The context ID of the chat. Adds an extra layer to the\n              local memory.",\n              value="",\n              advanced=True,\n          ),\n          MessageTextInput(\n              name="data_template",\n              display_name="Data Template",\n              value="{text}",\n              advanced=True,\n              info="Template to convert Data to Text. If left empty, it will be dynamically\n              set to the Data's text key.",\n          ),\n          BoolInput(\n              name="clean_data",\n              display_name="Basic Clean Data",\n              value=True,\n              advanced=True,\n              info="Whether to clean data before converting to string.",\n          ),\n      ]\n      outputs = [\n          Output(\n              display_name="Output Message",\n              name="message",\n              method="message_response",\n          ),\n      ]\n      def _build_source(self, id_: str | None,\n                        display_name: str | None, source: str | None) -> Source:\n          source_dict = {}\n          if id_: \n              source_dict["id"] = id_\n          if display_name: \n              source_dict["display_name"] =\n                  display_name\n          if source: \n              # Handle case where source is a ChatOpenAI object\n              if hasattr(source, "model_name"): \n                  source_dict["source"] = source.model_name\n              elif hasattr(source, "model"): \n                  source_dict["source"] = str(source.model)\n              else: \n                  source_dict["source"] = str(source)\n          return Source(**source_dict)\n\n      async def\n          message_response(self) -> Message: \n              # First convert the input to string if needed\n              text =\n                  self.convert_to_string()\n              # Get source properties\n              source, _, display_name, source_id =\n                  self.get_properties_from_source_component()\n              # Create or use existing Message object\n              if isinstance(self.input_value, Message) and not self.is_connected_to_chat_input():\n                  message = self.input_value\n                  # Update message properties\n                  message.text = text\n              # Preserve existing session_id from the incoming message if it exists\n              existing_session_id =\n                  message.session_id\n              else: \n                  message = Message(text=text)\n              existing_session_id = None\n              # Set message properties\n              message.sender =\n                  self.sender\n              message.sender_name = self.sender_name\n              # Preserve session_id from\n              incoming message, or use component/graph session_id\n              message.session_id = (\n                  self.session_id or existing_session_id or (self.graph.session_id if hasattr(self, "graph") else None)\n                  or ""\n              )\n              message.context_id = self.context_id\n              message.flow_id =\n                  self.graph.flow_id if hasattr(self, "graph") else None\n              message.properties.source =\n                  self._build_source(source_id, display_name, source)\n              # Store message if needed\n              if\n                  message.session_id and self.should_store_message:\n                  stored_message = await\n                  self.send_message(message)\n                  self.message.value = stored_message\n                  message =\n                  stored_message\n              # Set accumulated token usage from all upstream LLM vertices.\n              #\n              This must happen AFTER send_message() because streaming captures\n              # usage from chunks\n              and would overwrite accumulated totals.\n              if hasattr(self, "_vertex") and self._vertex is not\n                  None:\n                  accumulated_usage = self._vertex._accumulate_upstream_token_usage() # noqa:\n                  SLF001\n                  if accumulated_usage:\n                      message.properties.usage =\n                          accumulated_usage\n                  if self.should_store_message and message.get_id():

```

```

message = await self._update_stored_message(message)\n                await
self._send_message_event(message, id_=message.get_id())\n\n        self.status = message\nreturn message\n\n    def _serialize_data(self, data: Data) -> str:\n        \"\"\"Serialize Data object to\n        JSON string.\"\"\"\n        # Convert data.data to JSON-serializable format\n        serializable_data =\n        jsonable_encoder(data.data)\n        # Serialize with orjson, enabling pretty printing with indentation\n        json_bytes = orjson.dumps(serializable_data, option=orjson.OPT_INDENT_2)\n        # Convert bytes\n        to string and wrap in Markdown code blocks\n        return \"```\n        json\n        \"\"\" + json_bytes.decode(\"utf-8\")\n        + \"`\n        \"\"\"\n\n    def _validate_input(self) -> None:\n        \"\"\"Validate the input data and raise\n        ValueError if invalid.\"\"\"\n        if self.input_value is None:\n            msg = \"Input data cannot be\n            None\"\n            raise ValueError(msg)\n        if isinstance(self.input_value, list) and not all(\n            isinstance(item, Message | Data | DataFrame | str) for item in self.input_value\n        ):\n            invalid_types = [\n                type(item).__name__\n                for item in self.input_value\n                if not isinstance(item, Message | Data | DataFrame | str)\n            ]\n            msg = f\"Expected Data\n            or DataFrame or Message or str, got {invalid_types}\"\n            raise TypeError(msg)\n        if not\n            isinstance(\n                self.input_value,\n                Message | Data | DataFrame | str | list | Generator | \n                type(None),\n            ):\n            type_name = type(self.input_value).__name__\n            msg =\n            f\"Expected Data or DataFrame or Message or str, Generator or None, got {type_name}\"\n            raise TypeError(msg)\n\n    def convert_to_string(self) -> str | Generator[Any, None, None]:\n        \"\"\"Convert input data to string with proper error handling.\"\"\"\n        self._validate_input()\n        if\n            isinstance(self.input_value, list):\n                clean_data: bool = getattr(self, \"clean_data\", False)\n            return \"\n            \"\n            .join([safe_convert(item, clean_data=clean_data) for item in self.input_value])\n            if\n            isinstance(self.input_value, Generator):\n                return self.input_value\n            return\n            safe_convert(self.input_value)\n        ],\n        \"context_id\": {\n            \"_input_type\": \"MessageTextInput\",\n            \"advanced\": true,\n            \"display_name\": \"Context ID\",\n            \"dynamic\": false,\n            \"info\": \"The context ID of the chat. Adds an extra layer to the local memory.\",\n            \"input_types\": [\n                \"Message\"\n            ],\n            \"list\": false,\n            \"list_add_label\": \"Add More\",\n            \"load_from_db\": false,\n            \"name\": \"context_id\",\n            \"override_skip\": false,\n            \"placeholder\": \"\",\n            \"required\": false,\n            \"show\": true,\n            \"title_case\": false,\n            \"tool_mode\": false,\n            \"trace_as_input\": true,\n            \"trace_as_metadata\": true,\n            \"track_in_telemetry\": false,\n        },

```

```

    "type": "str",
    "value": ""
  },
  "data_template": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Data Template",
    "dynamic": false,
    "info": "Template to convert Data to Text. If left empty, it will be dynamically set to the
Data's text key.",
    "input_types": [
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "load_from_db": false,
    "name": "data_template",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_input": true,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "str",
    "value": "{text}"
  },
  "input_value": {
    "_input_type": "HandleInput",
    "advanced": false,
    "display_name": "Inputs",
    "dynamic": false,
    "info": "Message to be passed as output.",
    "input_types": [
      "Data",
      "JSON",
      "DataFrame",
      "Table",
      "Message"
    ],
    "list": false,
    "list_add_label": "Add More",
    "name": "input_value",
    "override_skip": false,

```

```

    "placeholder": "",
    "required": true,
    "show": true,
    "title_case": false,
    "trace_as_metadata": true,
    "track_in_telemetry": false,
    "type": "other",
    "value": ""
  },
  "sender": {
    "_input_type": "DropdownInput",
    "advanced": true,
    "combobox": false,
    "dialog_inputs": {},
    "display_name": "Sender Type",
    "dynamic": false,
    "external_options": {},
    "info": "Type of sender.",
    "name": "sender",
    "options": [
      "Machine",
      "User"
    ],
    "options_metadata": [],
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "toggle": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "str",
    "value": "Machine"
  },
  "sender_name": {
    "_input_type": "MessageTextInput",
    "advanced": true,
    "display_name": "Sender Name",
    "dynamic": false,
    "info": "Name of the sender.",
    "input_types": [
      "Message"
    ],
    "list": false,

```

```
"list_add_label": "Add More",
"load_from_db": false,
"name": "sender_name",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_input": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": "AI"
},
"session_id": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Session ID",
  "dynamic": false,
  "info": "The session ID of the chat. If empty, the current session ID parameter will be
used.",
  "input_types": [
    "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "session_id",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"should_store_message": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Store Messages",
  "dynamic": false,
```

```
    "info": "Store the message in the history.",
    "list": false,
    "list_add_label": "Add More",
    "name": "should_store_message",
    "override_skip": false,
    "placeholder": "",
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": true
  }
},
"tool_mode": false
},
"showNode": false,
"type": "ChatOutput"
},
"dragging": false,
"id": "ChatOutput-D1HOI",
"measured": {
  "height": 52,
  "width": 192
},
"position": {
  "x": 1661.799438406991,
  "y": 230.09731962265113
},
"selected": false,
"type": "genericNode"
}
],
"viewport": {
  "x": 136.651004958891,
  "y": 44.34285273498193,
  "zoom": 0.5977905174570449
}
},
"description": "Text Generation Meets Business Transformation.",
"endpoint_name": null,
"id": "efb6bca2-9013-4f40-928f-48711256f151",
"is_component": false,
"last_tested_version": "1.9.1",
```



```
"locked": false,  
"name": "Event generator POC",  
"tags": []  
}
```